THE UNIVERSITY OF ARIZONA
DEPARTMENT OF COMPUTER SCIENCE

# CSc 453: Fall 2025

## Assignment 1 Milestone 2

Start: Sept 11, 2025
Due: 11:59 PM, Sept 17, 2025

## 1. General

This assignment involves writing a parser for the G0 subset of C--.

## 2. Required functionality

### *2.1. Programming requirements*

#### Functionality

Your program should implement a function `parse()` that behaves as follows:

- It uses your scanner from Assignment 1 Milestone 1 to tokenize the input, which is read from **stdin**. **Note:** you should extend the scanner to keep track of line numbers in the input so that the parser can print out a line number when it encounters a syntax error.
- It checks whether the input token sequence is a syntactically legal program with respect to the syntax rules for the G0 language.
    - If the input is syntactically correct, your parser should exit silently with exit status 0.
    - If the input contains any syntax errors, your parser should print out an appropriate error message on **stderr** (see below) and exit with exit status 1. Your parser is not required to implement any sort of error recovery.

Your program should use the following driver file:

- driver.c -- a file containing the driver code for your parser.

  You may notice that the driver file mentions command-line arguments that control other aspects of a compiler, e.g., checking declarations, printing ASTs, and code generation. This is so that we can use the same driver file for future assignments as well. For this assignment, you can simply ignore these.

#### Makefile

You should submit a Makefile that provides (at least) the following targets:

- **make clean**:
  Deletes any object files ( *.o ) as well as the file 'compile'

- **make compile**:
  Compiles all the files from scratch and creates an executable file named 'compile'.

Note that the name of the executable is now different from that in Milestone 1 (**compile** rather than **scanner**).

### FIRST and FOLLOW sets

You can use the **gff** tool, which you used for Homework 1, to compute FIRST and FOLLOW sets for the grammar. Documentation on the use of gff is available here. You can access a gff executable in either of two ways:

1. it is available as an executable on lectura.cs.arizona.edu at **`/home/cs453/fall25/bin/gff`**; or
2. you can grab the source code (available as a zip file here) and build the executable yourself (if you opt for this and have problems building an executable, let me know).

### Error messages

Your error messages should satisfy the following requirements:

- Each error message should contain the string **`ERROR`**. The grading code will use this to identify error messages.
- Each error message should contain a string **`LINE`** *nnn*, where *nnn* is the line number where the error was detected. The grading code will use this to check whether errors are being detected at the right place. (Note that this requirement means that your scanner has to be extended to keep track of line numbers such that they are available to the parser.)

The case (upper/lower/mixed) of the strings ERROR and LINE mentioned above is not important: I used upper case just to make the strings conspicuous.

Apart from the two requirements above, you are free to craft your error message as you see fit. You should try to give error messages that are helpful to the user—imagine you were using your compiler "for real" and consider whether your error messages would be helpful for the user to understand and locate the problem.

## *2.2. Interface requirements*

Your program should be written to interface appropriately with the driver code mentioned above. To this end, your code should provide the following:

```
int parse();  /* the parser function */
```

# 3. Files you need to submit

- Makefile that supports the functionality described above;
- driver.c -- the driver file provided to you for this assignment;
- any additional files implementing your scanner and parser (including the scanner.h file from Milestone 1).

## 4. Gotchas to watch out for

There are two common sources of problems on this milestone:

1. Forgetting to exit with the correct exit code. Your parser should exit with a code of 0 for syntactically correct programs and 1 if there are any syntax errors.

2. A mismatch between the error messages produced by your compilers and the regular expression pattern the auto-grader uses to identify error messages. Here is the code used to determine whether a line in the output from your parser is an error message, and if so the line number it mentions. The line number is considered to be the first number following the string "**line**" (the regular expression used to extract the error line number is shown highlighted):

   ```
   # the variable line refers to a line of output generated by your parser
   line = line.lower()
   if "error" in line and "line" in line:
       match = re.match(".*line[^0-9]*(\d+)", line)
       if match is not None:
           err_line_num = match.group(1)     # <<< line no. of error
           break
   ```

   Information about Python regular expressions is available here. If your error messages follow the format specified in Section 2 (above), this code should identify error messages and the corresponding line numbers correctly.